Looped Threat Context Reasoning for Scalable Multiagent Target Defense

First Author, Second Author, and Third Author

***Abstract*—Multiagent target defense requires defenders to protect several assets from attackers. Because team and asset counts are rarely known in advance, a policy should train once on small engagements and transfer to larger ones. Transfer is hard because an attacker's threat depends on the target it approaches and on the teammates already covering it. We call this relation the threat context. We propose Looped Entity Attention with Fused reading (LEAF), which builds this context by looping one weight-shared entity-attention operator. A query formed from the defender's current state reads every round, the reads are fused, and recurrent memory enters only after reading. Training samples the number of rounds, so the policy can act after any round. In the SkyHAD aerial defense environment, policies trained with at most 10 defenders and 3 targets transfer without retraining to 50 defenders or 12 targets. LEAF has the lowest target damage in every test configuration, and in every test group even its weakest training run beats the best run of each baseline. Ablations confirm the roles of context construction, multi-round refinement, and memory placement. With a single round of reasoning, LEAF keeps nearly all of its protection, whereas a fixed-depth model collapses.**

***Note to Practitioners*—Teams of autonomous aircraft that guard airports, ports, or industrial sites must handle attacks whose size is unknown in advance. This work provides one control policy that is trained in small simulated engagements and then runs unchanged with several times more defenders, attackers, and protected sites. Every defender runs a copy of the same network on the positions, velocities, and status of the aircraft and sites it observes, and the network returns its next maneuver. Adding aircraft or sites needs no retraining, because all entities share the same parameters. The network first relates each attacker to the protected sites and nearby defenders, then interprets this picture from the defender's own situation, and only afterward uses its memory of earlier steps. Operators can also set how many reasoning rounds the network runs. A single round already keeps nearly all of the protection, so the controller can be matched to the onboard computer and the required decision rate without training a new model. The policy has so far been tested in simulation, where the defenders see every aircraft and site and the attackers follow a fixed rule. Field use calls for further tests with onboard sensors, communication delays, and attackers that adapt their tactics.**

***Index Terms*—Multiagent reinforcement learning, multi-target defense, scale generalization, entity attention, iterative reasoning, unmanned aerial vehicles.**

I. Introduction

DEFENDING stationary assets with a team of autonomous vehicles is a central problem for multirobot systems and aerial swarms [1]–[4]. The numbers of defenders, attackers, and protected targets are rarely known before a mission, so training a policy for every combination is impractical. A deployable controller should be trained once on small engagements and then run without retraining on much larger ones. We study this zero-shot scale generalization [5], [6].

Manuscript received Month DD, 2026. This work was supported by ...
The authors are with ... (e-mail: ...).

Placeholder for Fig. 1, single column.
The figure plan is given in the source comments.

Fig. 1. Threat context and the design of LEAF. (a) The same attacker approaching the same target is far more urgent when no teammate guards the target, so its own features cannot reveal its threat. (b) Where relations, memory, and depth enter. A single-read policy queries the entities once without relating them. An entity Transformer relates them in a fixed stack that also carries the recurrent memory. LEAF relates them with a weight-shared loop whose depth is set at deployment, reads every round with a current-state query, and lets memory enter only after the read.

Scale generalization is hard in target defense because threat is relational. Air-defense practice assesses a threat by its capability against the defended assets [7], and defense games show that the outcome depends on how defenders are arranged around the targets [2], [8]. The urgency of an attacker therefore depends on its motion, on the target it approaches, and on the teammates already covering that target. We call this joint relation the threat context. In Fig. 1(a), the same attacker approaches the same target in two scenes, and teammates guard the target only in the first. The features of the attacker are identical in both scenes, so no function of them can separate the two. A defender has to relate attackers to targets and targets to teammates before it decides [9], [10]. Larger engagements produce these relations in new combinations, so the computation that builds them must be shared across entities and remain valid for sets of any size [11], [12].

The context must also be read in the right way. The same context calls for different responses from a free defender and from one already engaged elsewhere, so each defender should

read it from its own state, as agent-specific attention does [6], [13]. Memory is needed as well, because attackers choose their targets privately and the scene is only partially observable [14], [15]. A recurrent state, however, moves into unseen regions when its inputs differ from those of training [16], [17], and a larger engagement supplies such inputs. If memory takes part in building or reading the threat context, this drift distorts the reading of the current scene.

A last question is how much relational reasoning a decision needs. One round of attention relates entities in pairs, while a guarded target is a relation among an attacker, its target, and nearby teammates. Further rounds compose such relations from earlier ones, as multiple hops do in memory networks [9], [18]. The required depth varies across inputs [19], [20], and a network trained at one depth degrades when it runs with fewer layers [20], [21]. A defense policy with a fixed depth therefore cannot shorten its reasoning to meet a tighter time budget.

Existing methods meet only part of these requirements. Assignment-based defenders, from differential games to learned allocation, compress the threat context into discrete defender-attacker pairs [4], [8], [22], [23]. Value factorization methods such as QMIX and DCG improve how utilities combine [24], [25], while each utility still comes from one read of the scene. Entity-based methods such as REFIL and SPECTra accept any team size [6], [26], yet they read the entities once without relating them first, as the top row of Fig. 1(b) shows. Entity Transformers such as TransfQMix relate entities over several layers [27], [28], but the same stack carries the recurrent memory and has a depth fixed in training.

Our key insight is that a defender's decision separates into three steps with different inputs. Building the threat context needs only the current entities. Reading it needs only the defender's current state. Inferring the hidden intent of attackers is the only step that needs history. A policy that generalizes across scales should give each step its own module, let history enter last, and train the context builder to work at any depth.

We propose LEAF (Looped Entity Attention with Fused reading). At each decision, LEAF refines the current entities with one weight-shared entity-attention operator applied in a loop, following the shared-depth computation of Universal and looped Transformers [29], [30]. A query formed only from the defender's current state reads the entities after every round, and learned weights fuse these reads. A recurrent network then combines the fused feature with the defender's memory and outputs action values. Training samples the number of rounds, so that every round count yields a usable policy [21]. Parameters are shared across entities, defenders, and rounds, and LEAF keeps the value factorization of REFIL for centralized training [6]. Unlike REFIL and SPECTra, LEAF relates entities before any defender reads them. Unlike TransfQMix, its scene processing never sees the recurrent state, and its depth can change after training, as the bottom row of Fig. 1(b) shows.

We evaluate LEAF in the SkyHAD aerial defense environment [31]. Policies trained with at most 10 defenders and 3 targets are deployed without retraining to up to 50 defenders or 12 targets. As teams grow, every learned baseline falls behind a rule-based assignment controller. LEAF has the lowest damage in every test configuration and outperforms all baselines by a wide margin. In every test group, even its weakest training run beats the best run of each baseline, and a baseline enlarged to the size of LEAF remains far behind. Single-factor ablations confirm that context construction, multi-round refinement, and memory placement each reduce damage, most clearly under extrapolation. A probe of the recurrent state confirms that memory drifts with scale while the current self state does not. Sampled-depth training keeps every execution depth usable. A fixed-depth model collapses when it runs with fewer rounds, whereas LEAF keeps nearly all of its protection with a single round.

The main contributions of this paper are as follows.

1) *Diagnosis.* We identify the threat context as the information on which zero-shot scale generalization in multi-target defense depends. We trace the scale failure of learned defenders to two design choices: reading entities before relating them, and letting a memory that drifts with scale take part in reading the scene.
2) *Method.* We propose LEAF, which separates context construction, current-state reading, and memory, and trains one weight-shared loop across sampled depths. A single policy then serves any team size and any execution depth.
3) *Evidence.* Trained on small engagements, LEAF transfers to teams and target sets several times larger with the lowest damage in every test configuration, and it keeps nearly all of its protection with a single round of reasoning.

Section II reviews related work, Section III formulates the problem, Section IV presents LEAF, Section V reports the experiments, and Section VI concludes the paper.

II. Related Work

We group prior work by the mechanism it uses to coordinate a changing number of defenders. Each family is represented among the baselines of Section V.

A. Assignment-Based Target Defense

Differential games derive defense strategies with guarantees, often by solving pairwise subgames and combining them through assignment [1], [8], [22], [32]. Task allocation makes the division of effort explicit [33]–[35]. Learning methods keep an allocation layer and learn when and how to assign, as in ALMA and hierarchical swarm defense [3], [4], [23], [36]. Our rule-based baseline re-solves an integer assignment of defenders to predicted attacker positions at every step. An assignment compresses the threat context into discrete pairs, which must be revised whenever attackers switch targets. LEAF keeps the context as entity representations and outputs maneuvers without an assignment.

B. Value Factorization and Explicit Coordination

Value decomposition connects individual utilities to a team objective through additive, monotonic, or pairwise structure [24], [25], [37]. These methods decide how utilities

combine, while each utility still comes from one read of the scene by its own agent. LEAF keeps a monotonic mixer for training and changes how the agent network forms its utilities.

C. Entity-Based Generalization Across Team Sizes

Set encoders and permutation-invariant networks process each entity with shared parameters [11], [12], [38], attention lets each agent weigh others by relevance [13], and population curricula reach large teams by training on them [39], [40]. REFIL factorizes values over random entity groups so that learned interactions recombine in new teams [6], and SPECTra pairs self-query attention with a mixer that scales with the team [26]. Both read the entity set once with the agent's own query, so relations among the other entities are not formed before the read. LEAF inserts a looped construction of these relations before the read.

D. Entity Transformers, Iterative Reasoning, and Memory

Entity Transformers apply several attention layers over entity tokens [27], [28]. UPDeT and TransfQMix carry history as a recurrent token in every layer, and their depth is fixed in training. Weight-shared iteration turns depth into repeated computation with constant parameters [19], [29], [30]. Stochastic depth, layer dropout, and early exits train networks that remain useful at several depths [20], [21], [41], [42], and jumping knowledge combines intermediate representations [43]. Recurrent memory is standard under partial observability [15], [44]. These ideas come from settings where test inputs resemble training inputs. LEAF combines a weight-shared loop, sampled depths, and fused reads inside one decision of a multiagent policy, and it keeps memory out of the loop.

III. Problem Formulation

This section formulates multi-target defense as a cooperative decision problem for the defenders. Section III-A introduces the simulation platform and the task. Section III-B casts the defenders' problem as a decentralized partially observable Markov decision process (Dec-POMDP), and Section III-C specifies the rule-based attacker that enters its transition. Section III-D states the zero-shot scale-generalization objective and derives what it requires of the policy in Section IV.

A. Simulation Platform and Task

SkyHAD [31] is an aerial attack-defense simulator in which red defenders protect stationary targets against blue attackers. We use its Damage task with acceleration-intent control. An episode contains a defender set $\mathcal{R}$, an attacker set $\mathcal{B}$, and a target set $\mathcal{T}$ with configuration $\mathbf{n} = (N_R, N_B, K)$ and $M = N_R+N_B+K$ entities. We write $\mathcal{R}_t$ and $\mathcal{B}_t$ for the aircraft alive at decision step t, $p_{j,t}$ and $v_{j,t}$ for the position and velocity of aircraft j, and p_k for the position of target k. As Fig. 2(a) shows, targets lie near the western edge of the scene, each defender starts around a random target, and the attackers enter from the east.

Decisions occur every $\Delta t = 1$ s. A defender acts by choosing an acceleration intent $a_{i,t}$ from a finite set $\mathcal{V}$ that holds the zero vector and a set of unit directions. The intent fixes the velocity reference

$$v_{i,t}^{\mathrm{ref}} = v_{i,t} + \bar{a}\,\Delta t\, a_{i,t}, \tag{1}$$

where $\bar{a}$ is the acceleration scale, and the aircraft tracks this reference within its speed and turn limits. The policy therefore chooses maneuvers, and the flight constraints remain in effect. An aircraft cannot hover and turns at a bounded rate, so a heading reversal takes many decisions.

Firing is automatic, and each aircraft fires at most once. A defender fires when a living attacker comes within the firing radius r_{f}, and an attacker fires when a target does. The firing aircraft is removed, and every enemy aircraft at distance ϱ, together with every target when the source is an attacker, takes the damage

$$\eta(\varrho) = \begin{cases} 1, & \varrho < r_0, \\ (r_1 - \varrho)/(r_1 - r_0), & r_0 \le \varrho < r_1, \\ 0, & \varrho \ge r_1. \end{cases} \tag{2}$$

Each interception thus costs one defender. Because aircraft have unit health, one interception can also remove several attackers that fly close together. Two aircraft whose paths pass within the collision distance during a step destroy each other, defenders included. Coordination therefore decides how many attackers each defender removes and how many defenders the team loses to collisions and redundant interceptions.

In the Damage task, targets never disappear and accumulate the damage they receive without an upper bound. An episode ends once no attacker remains, whether intercepted, destroyed in a collision, or expended on a target, and is truncated after $T_{\max} = 100$ decisions. Losing all defenders does not end the episode, so the surviving attackers keep adding damage.

B. Defender Dec-POMDP

With the attacker policy π_B of Section III-C fixed, defense in configuration $\mathbf{n}$ is the Dec-POMDP [14]

$$\mathcal{M}_{\mathbf{n}} = \big\langle \mathcal{R}, \mathcal{S}, \mathcal{A}, P^{\pi_B}, \{O_i\}, r, \gamma, \rho_0^{\mathbf{n}}, T_{\max} \big\rangle. \tag{3}$$

State and actions. The state $s_t \in \mathcal{S}$ holds the flight states and health of all aircraft, the accumulated target damage, the attackers' private target assignment z_t, and the decision index t. The reset law $\rho_0^{\mathbf{n}}$ samples the opening of Fig. 2(a). Each living defender selects an intent $a_{i,t} \in \mathcal{A} = \mathcal{V}$.

Transition. The attackers take the intents $\pi_B^{\mathrm{low}}(s_t)$ of their rule. Given these and the joint defender intent $\mathbf{a}_t$, the aircraft and targets evolve deterministically by the flight model and interaction rules of Section III-A. The assignment is then redrawn as $z_{t+1} \sim \pi_B^{\mathrm{up}}(\cdot \mid s_{t+1})$ at the command events of the rule and kept otherwise. Together with the reset law, these draws are the only randomness in the environment.

Observations. Defender i observes the entity set $o_{i,t} = O_i(s_t, \mathbf{a}_{t-1}) = \{x_{i,e,t}\}_{e \in \mathcal{E}_{i,t}}$ over the living aircraft and all targets, $\mathcal{E}_{i,t} = \mathcal{R}_t \cup \mathcal{B}_t \cup \mathcal{T}$. Each token holds the position and velocity of entity e, its type, its health if e is an aircraft, its accumulated damage if e is a target, and its

Figure placeholder. (a) Engagement in SkyHAD. (b) Decision structure of the defense problem.

Fig. 2. Multi-target defense in SkyHAD. (a) Targets lie near the western edge, defenders start around them, and attackers enter from the east. Shells mark the firing and damage radii of one target, rings show defender coverage, and dashed arrows show the attackers converging on the target their rule has privately selected. (b) Each defender maps its entity observations to an acceleration intent that the aircraft executes. Inside the environment, the attacker rule draws a target at command events and steers at every decision, and the drawn target stays hidden from the defenders.

previous intent if e is a defender. Positions and velocities are relative to defender i, except in its own token, which keeps them absolute. The observation is noiseless but omits z_t and the command timing. The history $\tau_{i,t} = (o_{i,0}, a_{i,0}, \dots, o_{i,t})$ therefore carries information that the current observation lacks. Fig. 2(b) summarizes this information structure.

Reward and objective. All defenders share the reward

$$r_t = -\Delta D_t = -\sum_{k\in\mathcal{T}} \sum_{b\in\mathcal{F}_t} \eta\big(\|p_{b,t} - p_k\|\big), \tag{4}$$

where $\mathcal{F}_t \subseteq \mathcal{B}_t$ holds the attackers that fire at step t. The reward counts simultaneous damage once per step for the team, and the episode reward is $R = -D$ with episode damage $D = \sum_t \Delta D_t$. Natural completion zeroes the successor value, while truncation at $T_{\max}$ retains it [45]. The defenders share one policy $\pi_\theta(a_{i,t} \mid \tau_{i,t})$ and maximize

$$J_{\mathbf{n}}(\theta) = \mathbb{E}\Big[\sum_{t=0}^{T-1} \gamma^t r_t \,\Big|\, \pi_\theta, \pi_B, \rho_0^{\mathbf{n}}\Big], \tag{5}$$

where $T \le T_{\max}$ is the stopping step. With $\gamma = 1$, $J_{\mathbf{n}}(\theta) = -\mathbb{E}[D \mid \mathbf{n}, \pi_\theta, \pi_B]$.

C. Rule-Based Attacker Policy

The attacker policy $\pi_B = (\pi_B^{\mathrm{up}}, \pi_B^{\mathrm{low}})$ is the fixed two-layer rule of SkyHAD, organized like the rule-based defender of Section V. Its upper layer assigns a target at command events, which occur at $t = 0$, every fifth decision, and every decision that follows a casualty. Its lower layer steers at every decision. The rule works in three steps.

Step 1, public threat evaluation. At a command event, the upper layer scores each target by the defender coverage around it and the mean distance the attackers must travel,

$$C_{k,t} = \sum_{i\in\mathcal{R}_t} \sigma_{i,t} \exp\big(-\|p_{i,t} - p_k\|/l_{\mathrm{c}}\big),$$
$$\bar{\varrho}_{k,t} = \frac{1}{|\mathcal{B}_t|} \sum_{b\in\mathcal{B}_t} \|p_{b,t} - p_k\|, \tag{6}$$

where $\sigma_{i,t} \in [0,1]$ is the health of defender i and l_{c} is a coverage length scale. Both scores use only public positions and health.

Step 2, concentrated stochastic assignment. The upper layer draws one target z_t from

$$\pi_B^{\mathrm{up}}(k \mid s_t) = \mathrm{softmax}_{k\in\mathcal{T}}\big(-2C_{k,t} - 0.25\,\bar{\varrho}_{k,t}/l_{\mathrm{tr}}\big), \tag{7}$$

with a travel length scale l_{tr}, and assigns every living attacker to it. The assignment is held until the next command event and stays private to the attackers.

Step 3, rush steering. At every decision, each attacker $b \in \mathcal{B}_t$ selects the intent best aligned with the direction to the assigned target,

$$a_{b,t}^B = \arg\max_{a\in\mathcal{V}} \; a^\top \frac{p_{z_t} - p_{b,t}}{\|p_{z_t} - p_{b,t}\|}, \tag{8}$$

and executes it through the same interface (1). This map is the lower layer π_B^{low}.

The rule makes the threat depend on the defenders' own layout, because the coverage term favors weakly covered targets. It concentrates the whole attacking force on one target at a time and can switch targets at every command event through a draw that the defenders do not observe.

D. Zero-Shot Scale Generalization

The configurations index a family $\{\mathcal{M}_{\mathbf{n}}\}$ that shares the dynamics, intents, attacker rule, and reward, and differs only in entity counts and reset law. A policy is trained on configurations drawn from P_{train} and deployed with frozen parameters on P_{test}, whose support contains larger teams, more targets, or both. Deployment quality is

$$\bar{D}_{\mathrm{test}}(\theta) = \mathbb{E}_{\mathbf{n}\sim P_{\mathrm{test}}}\, \mathbb{E}\big[D \mid \mathbf{n}, \pi_\theta, \pi_B\big], \tag{9}$$

while training maximizes only $\mathbb{E}_{\mathbf{n}\sim P_{\mathrm{train}}} J_{\mathbf{n}}(\theta)$. In Section V, P_{train} has at most ten aircraft per side and three targets, and P_{test} reaches fifty aircraft per side or twelve targets.

We learn π_θ by centralized training with decentralized execution (CTDE). A shared utility network Q_θ gives each defender the intent utilities $Q_\theta(\tau_{i,t}, \cdot) \in \mathbb{R}^{|\mathcal{V}|}$. During training, a monotonic mixer [24] combines them with a global entity table $g(s_t)$ that also omits z_t,

$$Q_{\mathrm{tot}}(s_t, \mathbf{a}_t) = f_{\mathrm{mix}}\big(\{Q_\theta(\tau_{i,t}, a_{i,t})\}_{i\in\mathcal{R}_t},\, g(s_t)\big),$$
$$\frac{\partial Q_{\mathrm{tot}}}{\partial Q_\theta(\tau_{i,t}, a_{i,t})} \ge 0, \tag{10}$$

so greedy individual intents also maximize Q_{tot}. At deployment each defender acts greedily on $Q_\theta(\tau_{i,t}, \cdot)$ without the mixer, and the utility network alone turns an entity set of any size into the intent of one defender. Zero-shot transfer therefore places three requirements on Q_θ.

Set-structured processing. The observation $o_{i,t}$ is a set whose size changes with $\mathbf{n}$ and with casualties. Q_θ must accept any number of tokens, give the same utilities under any reordering of the other entities, and share parameters across defenders [11], [12].

Relational threat context. Coverage enters the problem twice. Through (6) and (7) it shapes where the attackers go, and it decides whether an approaching attacker still needs a response, since teammates near its target can intercept it. The urgency of an attacker for defender i thus depends jointly on the attacker's approach to a target, the teammates covering that target, and the defender's own state. This attacker–target–teammate relation is the threat context of Section I. Q_θ must relate attacker, target, and teammate tokens before it extracts the feature on which defender i acts. The turn limit raises the cost of an error, since a defender committed to the wrong attacker needs many decisions to recover.

Memory after scene interpretation. The current scene shows which targets the attacker rule favors at its next command event. Only the history reveals which target its last draw selected, so Q_θ needs a recurrent memory [15]. Larger engagements contain more entities and more casualty-triggered reassignments, so a recurrent state trained on small engagements drifts into ranges it never visited. If that state conditioned the reading of the current scene, the drift would propagate into the threat context. Q_θ should therefore interpret the current scene from the current observation and add memory afterward.

Section IV builds LEAF on the entity-based factorization of REFIL [6] to meet these requirements. A shared looped attention operator constructs the threat context over the current entity set, a query formed from the defender's current state reads it, and recurrent memory enters after the read.

IV. Looped Entity Attention With Fused Reading

This section presents LEAF. Section IV-A gives the overall framework and its pseudocode, and Sections IV-B to IV-E describe its four modules.

A. Framework Overview

LEAF instantiates the utility network Q_θ of Section III-D on top of REFIL [6]. It rests on one observation. A defender's decision separates into three steps that draw on different information. Building the threat context draws only on the current entities, interpreting it for one defender adds only that defender's current state, and only inferring the hidden intent of the attackers needs history. LEAF gives each step its own module and lets history enter last. A fourth module trains the context builder so that it works at any depth.

Fig. 3 shows one decision of defender i. A shared encoder embeds the egocentric entity view, and the host path of REFIL pools it into a local feature. Module 1 refines the entity states for L rounds with one shared attention operator.

Algorithm 1 LEAF Decision and Sampled-Depth Training

Require: Actor and mixer parameters Θ, target copy $\bar{\Theta}$, replay buffer $\mathcal{D}$, maximum depth $L_{\max}$, weight λ

Decision of defender i at step t with depth L:
Compute $Z_{i,t}$, $\ell_{i,t}$, $E_{i,t}$, and $H_{i,t}^{(0)}$ by (11)
$q_{i,t} \leftarrow W_q E_{i,t}^{\mathrm{self}} + b_q$
for $r = 1$ **to** L **do**
  $H_{i,t}^{(r)} \leftarrow F^{m_{i,t}}(H_{i,t}^{(r-1)})$ by (12) ▷ Module 1
  Read $u_{i,t}^{(r)}$ with the fixed query by (13) ▷ Module 2
end for
Fuse the reads into $c_{i,t}^{[L]}$ by (14) ▷ Module 2
Update $h_{i,t}$ and output $Q_\theta(\tau_{i,t}, \cdot)$ by (15) ▷ Module 3
Training ▷ Module 4
for each collection round **do**
  Draw $L_k \sim \mathcal{U}\{1, \ldots, L_{\max}\}$ for each episode k
  Run each episode k with ϵ-greedy decisions at depth L_k and store it in $\mathcal{D}$
  for each gradient step **do**
    Sample a minibatch and $L \sim \mathcal{U}\{1, \ldots, L_{\max}\}$
    Unroll the online, target, and imagined branches with L rounds
    Update Θ by a gradient step on (17) at depth L
  end for
  Copy Θ to $\bar{\Theta}$ at a fixed interval
end for
Deployment: freeze Θ, choose L_{exec}, and act greedily without the mixer

Module 2 reads every round with a query formed from the defender's current self token and fuses the reads into a threat-context feature. Module 3 adds this feature to the host feature and passes the sum to a gated recurrent unit (GRU), which outputs the intent utilities. The loop restarts from the current observation at every decision, and only the GRU state carries memory across decisions. During training, Module 4 samples L, and the mixer of (10) combines the utilities into a team value. At deployment, the mixer is dropped and L becomes a compute budget. Algorithm 1 lists the procedure.

All parameters are shared across entities, defenders, and rounds, and every map acts on each entity separately or combines entities through masked attention. LEAF therefore accepts any number of entities, does not depend on their order [12], and needs neither a population-specific parameter nor an explicit population count. This meets the set-structured requirement of Section III-D. Modules 1 to 3 meet its other two requirements, and Module 4 answers the depth question raised in the Introduction.

B. Looped Threat-Context Construction

Defender i arranges its observation $o_{i,t}$ as a matrix $X_{i,t} \in \mathbb{R}^{M \times d_x}$ with one row per entity and its own token first. The self row keeps the absolute position and velocity of the defender, and the other rows give positions and velocities relative to it, which exposes distances and closing motion directly. The mask $m_{i,t} \in \{0,1\}^M$ marks destroyed aircraft and padding rows, which never serve as attention keys and

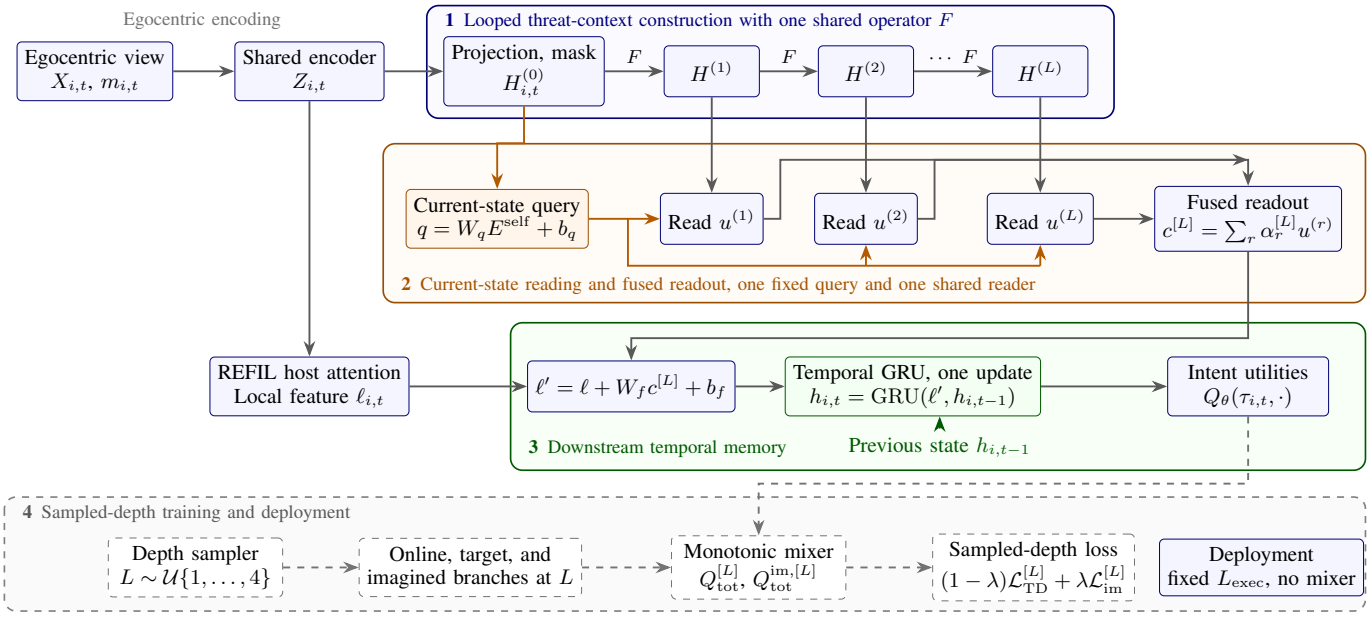


Fig. 3. Overview of LEAF at one decision of defender i. A shared encoder maps the egocentric entity view to tokens that feed the REFIL host path and the relational path. Module 1 initializes the context states $H^{(0)}$ from the current entities and refines them for L rounds with one shared operator F. Module 2 reads every round with a fixed query formed from the defender's current self token and fuses the reads into the threat-context feature $c^{[L]}$. Module 3 adds $c^{[L]}$ to the host feature, and the temporal GRU then produces the intent utilities. The GRU state is the only quantity that persists across decision steps. Module 4 samples L from one to four during training and applies the same depth to the online, target, and imagined branches. Deployment fixes L as a compute budget and runs without the mixer.

whose states $\text{Mask}_m(\cdot)$ sets to zero. A rowwise encoder ϕ_{enc}, an affine map with a rectified linear unit, feeds both paths:

$$Z_{i,t} = \phi_{\text{enc}}(X_{i,t}), \qquad \ell_{i,t} = \phi_{\text{host}}(Z_{i,t}, m_{i,t}),$$
$$E_{i,t} = \text{Lin}_g(Z_{i,t}), \qquad H_{i,t}^{(0)} = \text{Mask}_{m_{i,t}}(E_{i,t}). \tag{11}$$

Here ϕ_{host} is the self-query entity attention of REFIL with its output projection, Lin_g maps each token to the working width d, and $E_{i,t}^{\text{self}}$ is the self row of $E_{i,t}$.

Module 1 refines the context states with one operator F applied L times, $H_{i,t}^{(r)} = F^{m_{i,t}}(H_{i,t}^{(r-1)})$. Omitting i and t, one round is the residual attention block

$$\widetilde{H}^{(r)} = H^{(r-1)} + \text{MHA}_{\text{c}}^{m}\left(\text{LN}_a(H^{(r-1)}), H^{(r-1)}, H^{(r-1)}\right),$$
$$H^{(r)} = \text{Mask}_m\left[\widetilde{H}^{(r)} + \text{FFN}_{\text{c}}\left(\text{LN}_f(\widetilde{H}^{(r)})\right)\right], \tag{12}$$

where $\text{MHA}_{\text{c}}^{m}$ is multihead attention [46] over queries, keys, and values with $-\infty$ logits at excluded keys, and layer normalization and the two-layer feedforward network act on each row. The defender's own token takes part in the context like any other teammate.

The context is built before it is read because of a property shared by every read over individually embedded entities, such as the reads of REFIL and SPECTra [6], [26] or a read of $H^{(0)}$.

Proposition 1 (Selection without context): Let each row z_j of Z depend only on entity j and defender i, and consider the read $u = \text{MHA}^m(q, Z, Z)$. In every head, two permitted entities a and b receive attention weights with $w_a/w_b = \exp(s_a - s_b)$, where s_j depends only on q and z_j. The ratio is therefore identical in any two scenes that share q, z_a, and z_b, whatever the other entities are.

Proof: A head assigns $w_j = \exp(s_j)/\sum_k \exp(s_k)$, with $s_j = \langle W_Q q, W_K z_j \rangle / \sqrt{d_{\text{k}}}$ for key width d_{k} and the sum over permitted entities, so the normalizer cancels in w_a/w_b. ∎

Moving teammates toward or away from the targets of two attackers thus cannot change how such a read weighs one attacker against the other, and any dependence on coverage must be recovered later from the pooled vector. The ablation that reads $H^{(0)}$ in Section V tests this restriction directly. A read of $H^{(L)}$ with $L \geq 1$ is free of it, because by (12) every context state depends on all permitted entities. After one round, a target token can carry the teammates around it and an attacker token the target it approaches. After two, the attacker token can absorb the coverage-aware target token, so two rounds suffice for information to travel from teammate to target to attacker.

Sharing F across rounds makes depth a free variable of one computation. The parameter count does not depend on L, and every sampled depth trains the same operator on both the initial tokens and its own outputs, as with the shared transition of Universal Transformers [29]. Training thus favors a transformation that stays useful when applied to its own output, which is what execution at a different depth requires.

C. Current-State Reading and Fused Readout

Module 2 reads the context with a query formed from the defender's current self token, $q_{i,t} = W_q E_{i,t}^{\text{self}} + b_q$, fixed for all rounds of the decision. A reader with its own parameters, shared across rounds, produces one read per round,

$$u_{i,t}^{(r)} = \text{MHA}_{\text{r}}^{m_{i,t}}\left(q_{i,t}, H_{i,t}^{(r)}, H_{i,t}^{(r)}\right), \quad r = 1, \ldots, L. \tag{13}$$

The query carries the defender's own state and previous intent, so a free defender and one engaged elsewhere can attend to

different threats in the same scene. Because it ignores the loop states, every round answers the same question about a progressively refined context. It also ignores the temporal state, which Module 3 places downstream.

A linear scorer shared across rounds fuses the reads into the threat-context feature

$$\zeta_r = w_s^\top u_{i,t}^{(r)} + b_s, \qquad \alpha_r^{[L]} = \frac{\exp(\zeta_r)}{\sum_{k=1}^{L} \exp(\zeta_k)},$$
$$c_{i,t}^{[L]} = \sum_{r=1}^{L} \alpha_r^{[L]}\, u_{i,t}^{(r)}. \tag{14}$$

Because the scorer sees one read at a time, it serves any number of rounds, and the fusion weights still vary with the defender and the scene. Every depth L yields its own feature $c_{i,t}^{[L]}$, so stopping after any round, which we call an exit, gives a complete decision feature built from the same weights. As in jumping knowledge networks [43], a deeper exit keeps the early reads while adding refinement, and the feature never relies on the last round alone, whose token states repeated attention can homogenize [47]. All L rounds are computed, and the weights decide only how their reads are combined.

D. Downstream Temporal Memory

Module 3 adds the threat-context feature to the host feature and only then lets the temporal state enter:

$$\ell'_{i,t} = \ell_{i,t} + W_f c_{i,t}^{[L]} + b_f,$$
$$h_{i,t} = \mathrm{GRU}(\ell'_{i,t}, h_{i,t-1}), \qquad Q_\theta(\tau_{i,t}, \cdot) = W_o h_{i,t} + b_o. \tag{15}$$

The zero-initialized projection W_f, b_f lets training start from the host computation. The GRU updates once per decision, starts every episode from zero [15], and integrates the motion that reveals the private assignment of the attackers. At deployment, each defender selects the intent with the highest utility, and coordination arises from these intents without any assignment output.

For a fixed observation, mask, and depth, neither the query nor any quantity in (11) to (14) depends on $h_{i,t-1}$, so

$$\frac{\partial c_{i,t}^{[L]}}{\partial h_{i,t-1}} = 0. \tag{16}$$

History shapes the utilities through the GRU and never changes how the current scene is built or read. A recurrent state accumulated in a large engagement can leave the range of states seen in small training engagements, and Section V measures this drift. If such a state entered the query or the entity tokens, as the recurrent token of UPDeT and TransfQMix does [27], [28], it would change which entities the defender attends to. Under (16), the threat-context feature depends on the present observation alone.

E. Sampled-Depth Training and Deployment

Module 4 trains the actor through the learning interface of REFIL. The mixer f_{mix} of (10) uses attention-based hypernetworks over the entity table [6]. For each training sequence, REFIL also draws a random entity partition and forms within-group and cross-group masked views, whose auxiliary team value $Q_{\mathrm{tot}}^{\mathrm{im}}$ shares the target of the main branch. LEAF applies these masks to both feature paths. With team values $Q_{\mathrm{tot}}^{[L]}$ and $Q_{\mathrm{tot}}^{\mathrm{im},[L]}$ computed with L rounds, LEAF minimizes

$$\mathcal{L}(\Theta) = \mathbb{E}_{L\sim\mathcal{U}\{1,\dots,L_{\max}\}}\Big[(1-\lambda)\,\mathcal{L}_{\mathrm{TD}}^{[L]} + \lambda\,\mathcal{L}_{\mathrm{im}}^{[L]}\Big], \tag{17}$$

where $\mathcal{L}_{\mathrm{TD}}^{[L]}$ and $\mathcal{L}_{\mathrm{im}}^{[L]}$ are the mean squared errors of the two team values with respect to the target

$$y_t^{[L]} = r_t + \gamma\, \bar{Q}_{\mathrm{tot}}^{[L]}\big(s_{t+1}, \mathbf{a}_{t+1}^\star\big). \tag{18}$$

Here Θ holds the actor parameters θ and the mixer parameters, r_t is the training reward derived from (4), $\bar{Q}_{\mathrm{tot}}$ is the target network with parameters $\bar{\Theta}$, and $\mathbf{a}_{t+1}^\star$ collects the greedy intents of the online utilities. The bootstrap term is zero after natural termination, and we set $L_{\max} = 4$ and $\lambda = 0.5$.

Each parallel environment draws L when an episode starts and keeps it for the episode, so the temporal state evolves under one consistent policy. Each minibatch draws one L for its online, target, and imagined branches, so every temporal-difference error compares values from the same computation. Because Q-learning is off-policy, the two draws can be independent.

Every sampled depth supervises a complete policy. A shallow exit must already support good decisions, and a deeper exit must support them as well, so later rounds cannot undo what the first rounds built. Training at a fixed depth gives shallower exits no such signal. Sampled depth thus applies the idea of stochastic depth, LayerDrop, and early-exit training [20], [21], [41], [42] to the number of applications of one shared operator.

At deployment, LEAF freezes Θ, drops the mixer, and runs L_{exec} rounds. One round costs $O(M^2 d + M d^2)$ for each defender, so computation grows linearly with L_{exec} while the parameter count stays fixed, and the execution depth becomes a compute budget that needs no retraining. All experiments use $d_z = 128$, $d = d_h = 64$, four heads in every attention module, a feedforward width of $2d$, and $L_{\mathrm{exec}} = 4$ unless stated otherwise. The next property states when a change of this budget keeps the decisions.

Proposition 2 (Prefix-fusion stability): Fix one decision of defender i at step t, including its observation and entering temporal state, and compare depths $p < L$. Let

$$\beta = \sum_{r=p+1}^{L} \alpha_r^{[L]}, \qquad \xi = \max_{p<r\le L} \big\| u_{i,t}^{(r)} - c_{i,t}^{[p]} \big\|$$

be the fusion mass beyond round p and the largest distance from a later read to the depth-p feature. Then

$$\big\| c_{i,t}^{[L]} - c_{i,t}^{[p]} \big\| \le \beta\xi. \tag{19}$$

Let δ be the largest change of any utility $Q_\theta(\tau_{i,t}, a)$ between the two depths. Depth p selects the same greedy intent as depth L whenever the depth-L utility gap between the best and the second-best intents exceeds 2δ. If the map from c to $Q_\theta(\tau_{i,t}, \cdot)$ defined by (15) is κ-Lipschitz on the segment between $c_{i,t}^{[p]}$ and $c_{i,t}^{[L]}$, from the norm in ξ to the max norm, then $\delta \le \kappa\beta\xi$.

TABLE I
SCALE PROTOCOL. CONFIGURATIONS ARE WRITTEN (N, K) WITH $N = N_R = N_B$.

Set	Configurations (N, K)	Count
Training	$N \in \{4, 6, 8, 10\}$, $K \in \{1, 2, 3\}$, drawn per episode	–
ID	(4,2), (6,2), (8,2), (10,3)	4
K-OOD	(10,4), (10,6), (10,9), (10,12)	4
N-OOD	(15,2), (20,2), (25,2), (30,2), (40,2), (50,2)	6
Joint-OOD	(15,4), (15,6), (20,4), (20,6), (30,4), (30,6), (30,9), (30,12)	8
Extra	(5,2), (10,2)	2

Every evaluation configuration uses the same 300 initial scenes for all methods. The two extra configurations enter only the scale sweeps of Fig. 4 and the averages over all 24 configurations.

Proof: The first p rounds do not depend on L, so the reads and scores up to round p coincide at both depths, and $\alpha_r^{[p]} = \alpha_r^{[L]}/(1-\beta)$ for $r \le p$. Hence

$$c^{[L]} - c^{[p]} = \sum_{r=p+1}^{L} \alpha_r^{[L]} \big(u^{(r)} - c^{[p]}\big),$$

and the triangle inequality gives (19). If the best intent at depth L leads every other intent by more than 2δ, it still leads at depth p, since each utility moves by at most δ. The host feature and the entering temporal state are common to both depths, so the utilities differ only through c, and the Lipschitz condition with (19) gives $\delta \le \kappa\beta\xi$. ∎

The bound holds for every $p < L$, including $L > L_{\max}$, so it covers both truncating the loop and extending it beyond the training range. When the readout places little mass beyond round p and the late reads stay near the early fused feature, changing the depth barely moves the threat-context feature and keeps every decision with a clear utility gap. Section V measures this fusion mass and checks the condition on recorded decisions.

V. EXPERIMENTS

This section answers three questions. Does a policy trained once on small engagements protect targets better than existing methods when it is deployed at larger scales? Which designs of LEAF produce this generalization? Why do these designs work? Section V-A describes the setup, and Sections V-B to V-D answer the three questions in turn.

A. Experimental Setup

Task and scale protocol. All experiments use the Sky-HAD Damage task of Section III-A. N defenders protect K stationary targets against N attackers that follow the rule of Section III-C, and an episode ends when all attackers are destroyed or after 100 decisions. During training, every episode draws its own configuration with $N \in \{4, 6, 8, 10\}$ and $K \in \{1, 2, 3\}$. The frozen policies are then deployed on the configurations of Table I. These form an in-distribution (ID) group and three out-of-distribution (OOD) groups that enlarge the number of targets, the team size, or both, which we call K-OOD, N-OOD, and Joint-OOD. Two extra configurations inside the training range complete the scale sweeps. The largest deployments have five times as many aircraft per side or four times as many targets as the largest training configuration. Training adds a potential-based shaping term to the team reward (4), which leaves the optimal policy unchanged [48]. All reported results use the unshaped episode damage.

TABLE II
COMPARED METHODS. PARAMETERS COUNT THE NETWORKS USED AT DEPLOYMENT.

Method	Coordination mechanism	Target	Params
QMIX	Self-query entity attention, monotonic entity mixer	DQ	102k
REFIL	QMIX with randomly grouped imagined entity views	DQ	102k
DCG	Utilities and pairwise payoffs on a complete coordination graph	DQ	112k
ALMA	Learned target allocation and allocation-conditioned actor	DQ	283k
TransfQMix	Transformer actor and mixer with a memory token	TD(λ)	51k
SPECTra	Typed set projections, self-query read, set mixer	TD(λ)	47k
Rule	Integer assignment with predicted interception at every step	–	–
LEAF	Looped threat context on the REFIL host	DQ	176k

DQ denotes one-step double Q-learning with RMSprop, and TD(λ) denotes λ-returns with Adam, as in the original implementations. All learned methods carry recurrent memory.

Baselines. We compare LEAF with six learned baselines and a rule-based controller, summarized in Table II. Each represents one approach to scalable cooperative defense. QMIX [24] and REFIL [6] factorize the team value, and each defender reads the raw entities once with a query formed from its own state. We use QMIX in its entity-attention form with the entity mixer of REFIL. REFIL is the host of LEAF and thus its closest comparison. DCG [25] models pairwise coordination at the level of actions. ALMA [23] first allocates defenders to targets and then maneuvers, the decomposition common in defense automation. TransfQMix [28] and SPECTra [26] were designed for scalability. TransfQMix relates the entities and a memory token over several Transformer layers, and SPECTra encodes typed entity sets before a self-query read. Rule represents optimization-based engineering practice. At every step, it selects the attackers closest to the defenders, at most as many as there are living defenders, and solves a 0–1 integer program. The program assigns every defender to one selected attacker and covers every selected attacker while minimizing the total distance between the defenders and the attacker positions predicted one step ahead at constant velocity. Each defender then takes the acceleration intent closest to the velocity that intercepts its attacker. Like the attacker rule of Section III-C, Rule separates assignment from steering.

Common adaptation. All learned methods use the entity observation of Section III-B with the same features and masks, the same intent set, the same attackers, the same training distribution, and 10^6 environment steps per training run. Each method keeps its architecture and learning rule, and we replace only its input interface, action head, and masks. Each method also handles variable scale with its own mechanism. QMIX, REFIL, ALMA, and LEAF use the attention-based entity

TABLE III
ZERO-SHOT RESULTS ON THE FOUR TEST GROUPS. DAMAGE D GIVES THE MEAN AND SD OVER THREE TRAINING SEEDS, AND P AND CVAR GIVE SEED MEANS. THE BEST VALUE IN EACH COLUMN IS IN BOLD AND THE SECOND BEST IS UNDERLINED.

	Damage $D\downarrow$				Damage incidence P (%)$\downarrow$				Tail damage CVaR$\downarrow$			
Method	ID	K-OOD	N-OOD	Joint	ID	K-OOD	N-OOD	Joint	ID	K-OOD	N-OOD	Joint
QMIX	2.32±1.43	5.01±2.50	5.14±3.11	7.42±3.97	82.1	92.9	92.0	94.4	5.18	10.94	11.57	15.98
REFIL	0.72±0.05	1.80±0.30	2.21±0.28	3.31±0.30	44.4	60.4	67.0	73.2	2.79	6.18	7.81	10.66
DCG	2.03±0.44	5.84±1.92	5.02±2.66	8.09±3.91	85.2	96.4	94.8	97.9	4.99	12.07	11.77	17.32
ALMA	1.33±0.19	3.02±0.34	3.61±0.74	5.37±1.04	67.8	83.0	90.3	92.6	3.86	7.78	9.10	12.97
TransfQMix	1.53±0.28	3.95±0.50	4.02±0.97	6.63±1.20	68.5	88.1	89.5	93.9	4.39	9.63	10.84	15.87
SPECTra	2.48±0.21	6.74±0.53	5.05±0.59	9.84±0.49	90.9	98.9	95.6	99.3	5.71	13.30	12.09	19.49
Rule†	1.46±0.72	3.21±1.13	1.54±0.88	3.06±0.98	80.2	92.6	75.1	88.9	3.77	7.15	4.67	7.70
LEAF	**0.41**±0.10	**1.00**±0.23	**0.57**±0.23	**1.10**±0.10	**27.6**	**41.4**	**31.4**	**39.1**	**2.10**	**4.14**	**3.10**	**5.41**
LEAF vs. best	−43%	−45%	−63%	−64%	−38%	−31%	−53%	−47%	−25%	−33%	−34%	−30%

†Rule has no training seeds, and its SD is taken over scenes. Joint denotes Joint-OOD. CVaR is the mean damage of the worst 30 of the 300 episodes in each configuration. The last row compares LEAF with the best other method in each column.

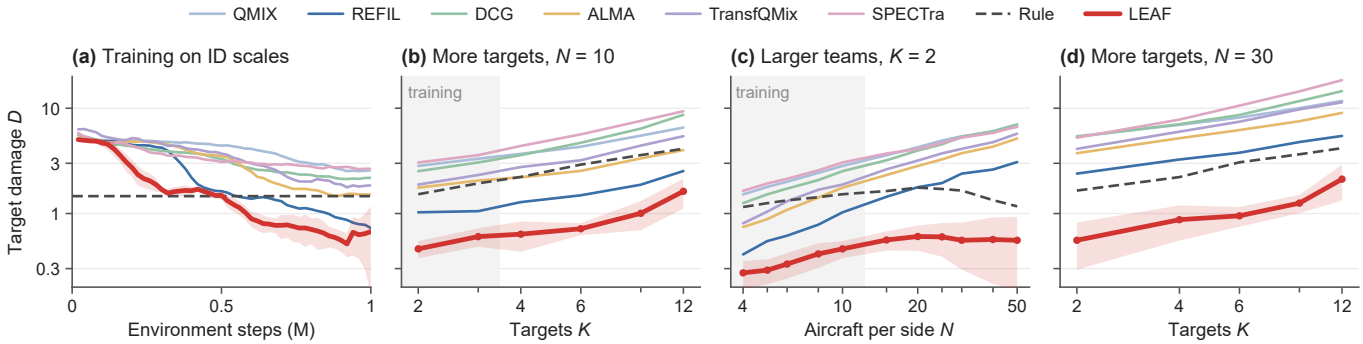


Fig. 4. Learning and zero-shot scale generalization. (a) Damage on the ID validation scenes during training, smoothed over five consecutive validation points. The dashed line marks the ID damage of Rule. (b)–(d) Damage of the frozen final policies as the number of targets grows at $N = 10$, as the team grows at $K = 2$, and as the number of targets grows at $N = 30$, beyond the training team size. Gray shading marks the training range. Each point averages 300 common scenes and three training seeds, and the red band shows one SD of LEAF over the seeds. Table III lists the SDs of all methods.

mixer of REFIL, SPECTra uses its set hypernetwork mixer, and TransfQMix uses its Transformer mixer. DCG builds a complete coordination graph over the deployed team, and ALMA treats each target as a subtask.

Training. Every method is trained with three seeds. Eight parallel environments collect data, and each collection round is followed by eight updates on minibatches of 32 episodes from a replay buffer of 5000 episodes. All methods use a discount of 0.99, gradient clipping at 10, hard target updates every 200 episodes, and ϵ-greedy exploration annealed from 1.0 to 0.05 over 10^5 steps, or 5×10^4 steps for DCG. LEAF, QMIX, REFIL, DCG, and ALMA use RMSprop with a learning rate of 5×10^{-4} and one-step double Q-learning targets [49]. SPECTra and TransfQMix keep Adam with a learning rate of 10^{-3} and λ-returns with $\lambda = 0.6$ from their implementations. LEAF uses the architecture of Section IV-E, samples L uniformly from one to four during training, and executes $L_{\text{exec}} = 4$ rounds unless stated otherwise. Every 2×10^4 steps, each run is evaluated on the four ID configurations with 25 episodes each from a separate scene pool. These 50 validation points produce the learning curves and play no role in model selection.

Evaluation and metrics. Evaluation is zero-shot. We take the final weights at the end of the budget, freeze them, select intents greedily, and apply no fine-tuning. Every model runs the same 300 initial scenes in every configuration. The attackers react to the defenders, so one initial scene can unfold differently under different policies, as closed-loop evaluation requires. The primary metric is the episode damage D of Section III-B, summed over all targets without capping. Two further metrics describe the quality of protection. The damage incidence P is the fraction of episodes in which at least one target is damaged. The conditional value at risk (CVaR) is the mean damage of the worst 10% of episodes in a configuration and measures tail risk. The mechanism analyses also report the fraction I of attackers that are intercepted and the fraction C of defenders lost to friendly collisions. For every trained model, we average over the scenes of a configuration and then over the configurations of a group with equal weight. We report the mean and the sample standard deviation (SD) over the three training seeds. Rule has no training seed, and its SD is taken over scenes. With three seeds per method, we state comparisons at the level of individual seeds, which readers can check directly [50].

B. Zero-Shot Scale Generalization

We first ask whether a policy trained once on small engagements protects targets better than existing methods when it is deployed at larger scales. Table III and Fig. 4 answer this question.

TABLE IV

SINGLE-FACTOR ABLATIONS. EACH ROW OF THE MAIN CHAIN CHANGES ONE DESIGN OF THE ROW ABOVE, AND EACH SIDE ROW CHANGES ONE DESIGN OF THE PARENT IT NAMES. DAMAGE D GIVES THE MEAN AND SD OVER THREE TRAINING SEEDS, WITH THE CHANGE RELATIVE TO THE ROW ABOVE IN PARENTHESES. THE BEST VALUE IN EACH COLUMN IS IN BOLD.

	Variant	Change	Train L	Query	ID	K-OOD	N-OOD	Joint-OOD
0	Direct read	Reads the raw entities $H^{(0)}$	–	memory	0.93±0.21	1.96±0.46	2.02±0.45	3.20±0.69
1	One round	Adds one context round	1	memory	0.61±0.27 (−34%)	1.20±0.40 (−39%)	1.36±0.88 (−33%)	2.16±1.01 (−33%)
2	Fixed depth	Refines for four rounds	4	memory	0.41±0.20 (−33%)	1.00±0.66 (−17%)	0.66±0.18 (−51%)	1.27±0.23 (−41%)
3	Memory query	Samples the training depth	1–4	memory	0.53±0.23 (+29%)	1.53±0.73 (+52%)	1.00±0.62 (+51%)	1.95±1.23 (+54%)
4	**LEAF**	Moves memory out of the query	1–4	current	**0.41**±0.10 (−23%)	**1.00**±0.23 (−35%)	0.57±0.23 (−42%)	**1.10**±0.10 (−44%)
	Unshared rounds	Fixed depth, untied rounds	4	memory	0.48±0.18	1.13±0.16	**0.54**±0.16	1.23±0.25
	Last read	Memory query, last read only	1–4	memory	0.43±0.08	1.02±0.10	0.65±0.16	1.27±0.20
	No host	Memory query, no REFIL host	1–4	memory	0.51±0.14	1.77±0.62	1.26±0.65	2.32±0.89
	REFIL-matched	REFIL widened to 176k parameters	–	–	0.91±0.17	2.49±0.28	2.27±0.31	4.17±0.76

All variants execute four rounds except One round, which executes one, and Direct read and REFIL-matched, which have no context loop. A memory query adds the previous recurrent state, with a stopped gradient, to the current self token that forms the query.

LEAF has the lowest mean damage in all four groups and in each of the 24 evaluation configurations. Relative to the strongest competitor in each group, it reduces damage by 43% in ID, 45% under target-count extrapolation, 63% under team-size extrapolation, and 64% under joint extrapolation. The advantage holds for every training run. In every group, the worst of the three LEAF models incurs less damage than the best model of each learned baseline and than Rule. The ordering repeats on an independent pool of 300 scenes per configuration, where the damage averaged over all 24 configurations is 0.79 for LEAF, 2.09 for REFIL, 4.25 for TransfQMix, and 6.30 for SPECTra.

In Fig. 4(c), the damage of LEAF stays between 0.56 and 0.61 as teams grow from 15 to 50 aircraft per side, whereas every learned baseline degrades steadily. The damage of REFIL, for example, rises from 1.44 to 3.08. More attackers create more combinations of attackers, targets, and teammates that a defender must tell apart. By Proposition 1, a read of individually embedded entities weighs each attacker by its own features and the query, so it cannot favor an attacker because the attacker's target is unguarded. LEAF relates attackers to targets and teammates before any defender reads the scene, and its damage barely changes as the team grows. Rule solves its assignment anew at every scale and also stays nearly flat. Under team-size and joint extrapolation, it incurs less damage than every learned baseline. In Fig. 4(b) and (d), more targets raise the damage of every method, because the attackers then have more targets to choose from. LEAF remains the lowest at every target count, and this holds when the team is already three times the training size.

The protection metrics in Table III show the same advantage at the level of episodes. LEAF keeps every target undamaged in 59% to 72% of episodes. No other method exceeds 56% in any group, and Rule stays below 25%. The mean damage of the worst 10% of episodes is 25% to 34% lower for LEAF than for the strongest competitor in each group. Rule is the strongest competitor on average damage at large scales, yet it lets at least one target be damaged in 75% to 93% of episodes. Assigning attackers by distance gives every attacker a pursuer, but it cannot tell which attacker threatens an uncovered target. LEAF weighs each attacker by its threat context and leaves fewer targets exposed.

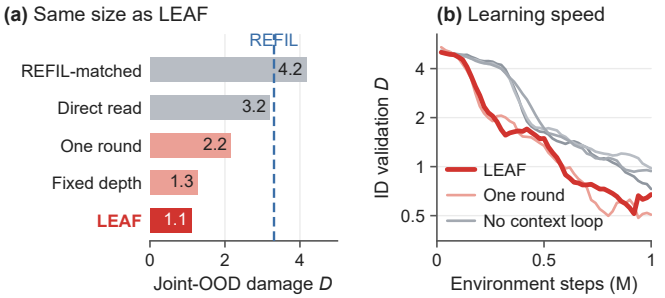


Fig. 5. Organization versus size. (a) Joint-OOD damage of the models that share the size of LEAF, 176k parameters. Gray bars mark models that read the entities without a context loop, and red bars models that build the context in a loop. The dashed line marks REFIL, which has 102k parameters. (b) ID validation damage during training, smoothed over five validation points, with REFIL, REFIL-matched, and Direct read drawn in gray. All values are means over three training seeds.

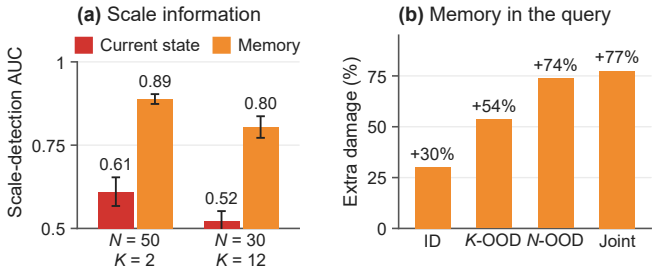


Fig. 6. Memory after reading. (a) AUC of a linear classifier that separates decisions in a shifted configuration from decisions at $(N, K) = (10, 2)$, using either the current self features that form the query of LEAF or its recurrent state h_{t-1}. Bars start at the chance level of 0.5, and error bars show one SD over three training seeds. Each configuration contributes ten scenes per model. (b) Extra damage of Memory query, which feeds the recurrent state into the query, relative to LEAF in each group. Joint denotes Joint-OOD.

LEAF also learns faster. In Fig. 4(a), its validation damage first falls below 1.0 after about 0.56×10^6 steps. REFIL needs about 0.86×10^6 steps, and no other learned baseline reaches this level within the budget.

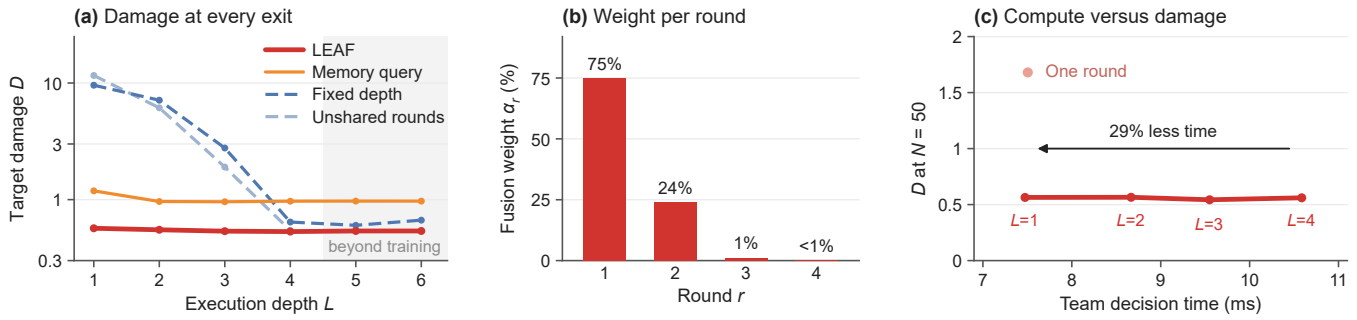


Fig. 7. Execution at any depth. (a) Closed-loop damage at execution depths one to six, averaged over $(N, K) = (10, 2)$, $(30, 2)$, and $(50, 2)$ with 300 scenes each and over three training seeds. Solid lines mark sampled-depth training and dashed lines training at four rounds only. Unshared rounds has no parameters beyond its fourth round, and LEAF comes from a separate depth evaluation with the same protocol. (b) Fusion weight of each round in normal four-round execution of LEAF, averaged over $(10, 2)$, $(30, 2)$, and $(30, 12)$. (c) Full-team decision time and damage at $(50, 2)$ for LEAF executed with one to four rounds and for One round. Times come from 20 closed-loop episodes on the workstation described in Section V-D, and damage values are means over three training seeds.

C. Ablation Study

We next retrain variants of LEAF to identify which designs produce this generalization. All variants share the training budget, seeds, and evaluation of LEAF. Their saved configurations show that consecutive rows of the main chain in Table IV differ in exactly one switch, so the chain adds the designs of Section IV one at a time. Direct read reads the raw entities $H^{(0)}$. One round builds the context for one round before reading, and Fixed depth refines it for four rounds with training at four rounds only. Memory query samples the training depth, and LEAF finally removes the recurrent state from the query. Every variant before LEAF feeds the previous recurrent state into the query with a stopped gradient, so the last step isolates the placement of memory.

Context construction is the foundation. A single context round removes 33% to 39% of the damage of Direct read in every group, with the query and training otherwise unchanged. Direct read is exactly the read that Proposition 1 restricts, so moving teammates toward or away from a target cannot shift its attention between attackers. After one round, each attacker token already carries information about the target it approaches and the teammates nearby, and the read can weigh attackers by their context.

Refining the context for four rounds lowers damage further, by 33% in ID and by 51% and 41% under team-size and joint extrapolation, and all three seeds improve in both of these groups. In small engagements, one round already captures the relations that matter. With more entities, whether a target is covered depends on relations that a second round composes from the first, as Section IV-B explains. Multi-round refinement therefore pays off most when the scale grows.

Removing the recurrent state from the query lowers damage by 23% in ID and by 35% to 44% in the extrapolation groups. Seen from LEAF, feeding memory into the query raises damage by 30% in ID and by 54% to 77% under extrapolation, and the spread across seeds grows with it. Under joint extrapolation, the SD rises from 0.10 for LEAF to 1.23 for Memory query. Memory placement thus matters most where deployment departs furthest from training, and Section V-D traces this effect to drift of the recurrent state.

Sampling the training depth raises the four-round damage within the memory-query family, mostly through one of the three training runs. LEAF also samples its depth and matches or improves on Fixed depth in all four groups. The value of sampled depth lies in the shallower exits, which Section V-D examines. Untying the rounds in Unshared rounds or reading only the last round in Last read leaves the four-round damage close to that of LEAF, and these choices matter when the execution depth changes. Removing the REFIL host in No host raises damage in all three extrapolation groups, so LEAF builds on the entity grouping of its host.

D. Mechanism Analysis

Finally, we examine why LEAF works through three properties of its computation. The gain comes from how the computation is organized, reading the scene from the current state keeps its interpretation stable as memory drifts, and sampled-depth training makes every execution depth a usable policy.

Organization over size. REFIL-matched widens the actor of REFIL with a residual multilayer perceptron to within 0.4% of the parameter count of LEAF. Its damage is 2.3 to 3.9 times that of LEAF and higher than that of REFIL in all four groups. Fig. 5(a) compares the models that share the size of LEAF. Their joint-extrapolation damage spans almost a factor of four, and every model that builds the context in a loop incurs less damage than REFIL. Size does not predict damage across methods either. ALMA has as many parameters as Unshared rounds and more than four times its joint-extrapolation damage. Context construction also speeds up learning. In Fig. 5(b), One round learns at the pace of LEAF, whereas Direct read and REFIL-matched follow REFIL. The trained policy relies on this context. When the frozen LEAF policy reads the raw entities $H^{(0)}$ in place of its context, damage rises in all nine combinations of model and configuration that we tested, with ten scenes each.

Memory after reading. Section IV-D argued that a recurrent state accumulated in a larger engagement can leave the range

seen in training. We test this with a probe [51]. For each training seed, a classifier separates decisions in a shifted configuration from decisions at $(N, K) = (10, 2)$, using either the recurrent state h_{t-1} of LEAF or the current self features from which its query is formed. In Fig. 6(a), a linear classifier identifies the deployment scale from the recurrent state with an area under the receiver operating characteristic curve (AUC) of 0.89 at $(50, 2)$ and 0.80 at $(30, 12)$. From the current self features it reaches only 0.61 and 0.52. Gradient-boosted trees give the same picture, and every seed agrees. Memory learned in small engagements thus moves into new regions as the scale grows, while the current self state stays close to the training distribution. The query is a deterministic function of the self features, so it carries no more scale information than they do.

Feeding memory into the query carries this drift into the reading of the scene. Fig. 6(b) shows that the extra damage of Memory query grows from 30% in ID to 77% under joint extrapolation, and Table IV shows that its seeds spread further apart. Team behavior follows the same order. In the four configurations with protection records, Memory query lies between REFIL and LEAF on interception, damage incidence, tail damage, and friendly collisions in all 16 comparisons. Memory remains useful downstream. Resetting the recurrent state of the frozen LEAF policy at every step raises damage in eight of nine combinations of model and configuration. LEAF therefore keeps memory for selecting intents and keeps it out of the step that reads the threat context.

Every exit is a usable policy. In normal four-round execution, the first two rounds receive about 99% of the fusion weight in every configuration we examined, and the first round alone about three quarters, as Fig. 7(b) shows. The split varies across defenders, with an SD of 11 to 14 percentage points among the defenders of the same state, so each defender draws on the rounds in its own way. Proposition 2 links this small fusion mass beyond the second round to stable decisions. We execute two rounds instead of four in the same states for 48,705 decisions at $(10, 2)$, $(50, 2)$, and $(30, 12)$, with ten scenes per model. The greedy intent changes in only 1.1% to 1.7% of the decisions. For 79% of them, the utility gap exceeds twice the largest utility change δ between the two depths, the condition of the proposition, so these decisions keep their intent. The decisions that do change lie mostly in the lowest tenth of utility gaps, 78% to 81% of them, where the two best intents are nearly equivalent.

Closed-loop execution confirms the same picture. Over all 24 configurations, executing two rounds instead of four changes the damage of LEAF by about 1%, and a single round by about 2%. Executing five or six rounds, beyond the training range, changes it by less than 0.1%. Sampled-depth training is what keeps the shallow exits usable. Memory query and Fixed depth share the architecture and the query and differ only in how the training depth is chosen. In Fig. 7(a), the two sampled-depth models stay flat from one to six rounds. Fixed depth executed for one round incurs 15 times its own four-round damage. Five of the six models trained at four rounds collapse in this way, whereas all six sampled-depth models stay within a factor of 1.5 of their four-round damage.

A shorter exit saves computation at deployment. We time the policies on a workstation with an NVIDIA GeForce RTX 5070 Ti GPU and an Intel Core i7-14700KF CPU, using PyTorch 2.13 with CUDA 13.0. A full-team decision for 50 defenders takes 7.5 ms with one round and 10.6 ms with four, two orders of magnitude below the decision interval of 1 s. An operator can therefore cut the decision time by 29% without retraining, at a cost of about 2% in average damage. Fig. 7(c) adds that LEAF executed for one round runs as fast as One round, the model trained with a single round, and incurs about one third of its damage at $(50, 2)$.

VI. Conclusion

This paper studied zero-shot scale generalization in multi-agent target defense. We identified the threat context, the relation among attackers, their targets, and the teammates that cover those targets, as the information a defender must build before it acts. LEAF builds this context by looping one weight-shared entity-attention operator over the current entities. A query formed from the defender's current state reads every round, learned weights fuse the reads, and recurrent memory enters only after the read. Sampled-depth training makes every round count a usable policy.

Trained with at most 10 defenders and 3 targets and deployed without retraining to 50 defenders or 12 targets, LEAF had the lowest damage in every test configuration, and even its weakest training run outperformed the best run of every baseline. It also kept every target undamaged in more episodes and reduced the damage of the worst episodes. Single-factor ablations showed that context construction, multi-round refinement, and current-state reading each reduce damage, and the last two matter most under extrapolation. A baseline enlarged to the size of LEAF remained far behind, so the gain comes from how the computation is organized. A probe of the recurrent state showed that memory drifts with scale while the current self state does not, which explains why memory belongs after the read. Under sampled-depth training, LEAF placed about 99% of the fusion weight on the first two rounds. A single round therefore kept nearly all of the protection at 29% less decision time, whereas a model trained at a fixed depth collapsed when shortened. For automated defense systems, these results mean that one trained policy can serve teams of different sizes and onboard computers of different speeds.

Several limits of this study point to future work. All results come from simulation, in which defenders observe every entity without noise, the attackers follow a fixed rule, and each method was trained with three seeds. The tests reached 50 aircraft per side or 12 targets. Field deployment calls for onboard sensing with limited range, communication delays, and attackers that adapt their tactics, and it would benefit from evaluation against learned attackers. The execution depth is currently set by the operator, and a policy that chooses its number of rounds for each decision could save further computation. Extending the threat context to heterogeneous defenders and to targets of different value is another direction.

ACKNOWLEDGMENT

The authors used AI assistants based on large language models, accessed through the Cursor editor, to draft and revise the text of all sections, to lay out the tables and the method diagram, and to write the scripts that plot the experimental figures from the recorded results. The authors checked all content against the experimental records and take full responsibility for the paper.

REFERENCES

[1] E. Garcia, D. W. Casbeer, and M. Pachter, "Design and analysis of state-feedback optimal strategies for the differential game of active defense," *IEEE Transactions on Automatic Control*, vol. 64, no. 2, pp. 553–568, 2019.

[2] D. Shishika, J. Paulos, and V. Kumar, "Cooperative team strategies for multi-player perimeter-defense games," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2738–2745, 2020.

[3] Z. Zhao, J. Chen, B. Xin, L. Li, K. Jiao, and Y. Zheng, "Learning scalable task assignment with imperative-priori conflict resolution in multi-UAV adversarial swarm defense problem," *Journal of Systems Science and Complexity*, vol. 37, no. 1, pp. 369–388, 2024.

[4] Q. Wu, K. Liu, L. Chen, and J. Lü, "Hierarchical reinforcement learning for swarm confrontation with high uncertainty," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 8630–8644, 2025.

[5] R. Kirk, A. Zhang, E. Grefenstette, and T. Rocktäschel, "A survey of zero-shot generalisation in deep reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 76, pp. 201–264, 2023.

[6] S. Iqbal, C. A. S. de Witt, B. Peng, W. Boehmer, S. Whiteson, and F. Sha, "Randomized entity-wise factorization for multi-agent reinforcement learning," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 4596–4606.

[7] J. N. Roux and J. H. van Vuuren, "Threat evaluation and weapon assignment decision support: A review of the state of the art," *ORiON*, vol. 23, no. 2, pp. 151–187, 2007.

[8] E. Garcia, D. W. Casbeer, A. Von Moll, and M. Pachter, "Multiple pursuer multiple evader differential games," *IEEE Transactions on Automatic Control*, vol. 66, no. 5, pp. 2345–2350, 2021.

[9] P. W. Battaglia, J. B. Hamrick, V. Bapst *et al.*, "Relational inductive biases, deep learning, and graph networks," arXiv:1806.01261, 2018.

[10] V. Zambaldi, D. Raposo, A. Santoro *et al.*, "Deep reinforcement learning with relational inductive biases," in *International Conference on Learning Representations*, 2019.

[11] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3391–3401.

[12] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 3744–3753.

[13] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 2961–2970.

[14] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*, ser. SpringerBriefs in Intelligent Systems. Cham, Switzerland: Springer, 2016.

[15] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[16] A. Lamb, A. Goyal, Y. Zhang, S. Zhang, A. Courville, and Y. Bengio, "Professor forcing: A new algorithm for training recurrent networks," in *Advances in Neural Information Processing Systems*, vol. 29, 2016, pp. 4601–4609.

[17] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 1171–1179.

[18] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus, "End-to-end memory networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2440–2448.

[19] A. Graves, "Adaptive computation time for recurrent neural networks," arXiv:1603.08983, 2016.

[20] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 3301–3310.

[21] A. Fan, E. Grave, and A. Joulin, "Reducing transformer depth on demand with structured dropout," in *International Conference on Learning Representations*, 2020.

[22] M. Chen, Z. Zhou, and C. J. Tomlin, "Multiplayer reach-avoid games via pairwise outcomes," *IEEE Transactions on Automatic Control*, vol. 62, no. 3, pp. 1451–1457, 2017.

[23] S. Iqbal, R. Costales, and F. Sha, "ALMA: Hierarchical learning for composite multi-agent tasks," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 7155–7166.

[24] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4295–4304.

[25] W. Boehmer, V. Kurin, and S. Whiteson, "Deep coordination graphs," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 980–991.

[26] H. Park, B. Seong, and S.-K. Ko, "Scalable multi-agent reinforcement learning with permutation-free networks," *IEEE Access*, vol. 14, pp. 95 650–95 664, 2026.

[27] S. Hu, F. Zhu, X. Chang, and X. Liang, "UPDeT: Universal multi-agent reinforcement learning via policy decoupling with transformers," in *International Conference on Learning Representations*, 2021.

[28] M. Gallici, M. Martin, and I. Masmitja, "TransfQMix: Transformers for leveraging the graph structure of multi-agent reinforcement learning problems," in *Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems*, 2023, pp. 1679–1687.

[29] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser, "Universal transformers," in *International Conference on Learning Representations*, 2019.

[30] A. Giannou, S. Rajput, J. yong Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos, "Looped transformers as programmable computers," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 11 398–11 442.

[31] "SkyHAD," Software, version 4.2.0, 2026, [Online]. Available: https://github.com/Sailero/SkyHAD.

[32] R. Yan, X. Duan, Z. Shi, Y. Zhong, and F. Bullo, "Matching-based capture strategies for 3D heterogeneous multiplayer reach-avoid differential games," *Automatica*, vol. 140, p. 110207, 2022.

[33] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[34] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.

[35] A. Miele, M. Lippi, and A. Gasparri, "A distributed framework for integrated task allocation and safe coordination in networked multi-robot systems," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 11 219–11 238, 2025.

[36] X. Li, Z. Li, X. Yang, X. Yu, and J. Qiu, "Multi-agent target defense differential game: A hierarchical recognition-allocation-execution learning approach," *International Journal of Robust and Nonlinear Control*, vol. 35, no. 16, pp. 6868–6879, 2025.

[37] P. Sunehag, G. Lever, A. Gruslys *et al.*, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems*, 2018, pp. 2085–2087.

[38] J. Hao, X. Hao, H. Mao *et al.*, "Boosting multiagent reinforcement learning via permutation invariant and permutation equivariant networks," in *International Conference on Learning Representations*, 2023.

[39] Q. Long, Z. Zhou, A. Gupta, F. Fang, Y. Wu, and X. Wang, "Evolutionary population curriculum for scaling multi-agent reinforcement learning," in *International Conference on Learning Representations*, 2020.

[40] W. Wang, T. Yang, Y. Liu *et al.*, "From few to more: Large-scale dynamic multiagent curriculum learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 5, 2020, pp. 7293–7300.

[41] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep networks with stochastic depth," in *Computer Vision – ECCV 2016*, ser.

Lecture Notes in Computer Science, vol. 9908. Cham: Springer, 2016, pp. 646–661.

[42] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proceedings of the 23rd International Conference on Pattern Recognition*, 2016, pp. 2464–2469.

[43] K. Xu, C. Li, Y. Tian, T. Sonobe, K. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 5453–5462.

[44] T. Phan, F. Ritz, P. Altmann *et al.*, "Attention-based recurrence for multi-agent reinforcement learning under stochastic partial observability," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 27 840–27 853.

[45] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4045–4054.

[46] A. Vaswani, N. Shazeer, N. Parmar *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[47] Y. Dong, J.-B. Cordonnier, and A. Loukas, "Attention is not all you need: Pure attention loses rank doubly exponentially with depth," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 2793–2803.

[48] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning*, 1999, pp. 278–287.

[49] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.

[50] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 29 304–29 320.

[51] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," in *International Conference on Learning Representations, Workshop Track*, 2017.